

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-АВТОТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ALIEXPRESS

Команда:

Степанов; Шакуров;
Кислица; Миридаштаки

Руководитель

А.М. Шевелева

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Ф. Миридаштаки

Группа: 4343

Тема практики: UI-автотестирование веб-приложения AliExpress

Задание на практику:

Разработать набор UI-автотестов для проверки пользовательских сценариев веб-приложения AliExpress в рамках общего тестового проекта, построенного на основе паттерна Page Object Model. В рамках индивидуальной части составить чек-лист сценариев, реализовать автоматизированные тесты сортировки результатов поиска по цене, очистки строки поиска, переключения языка интерфейса, перехода на страницу продавца и выбора количества товара в карточке. Разработать вспомогательные классы элементов Item и Link, а также класс страницы SellerPage. Провести контрольный запуск реализованных тестов и оформить результаты в отчёте.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент	_____	Ф. Миридаштаки
---------	-------	----------------

Руководитель	_____	А.М. Шевелева
--------------	-------	---------------

АННОТАЦИЯ

В отчёте представлены результаты учебной практики по разработке UI-автотестов для веб-приложения AliExpress. Составлен чек-лист из 15 пользовательских сценариев, спроектирована архитектура на основе Page Object Model и реализованы пять автоматизированных проверок: сортировка результатов поиска по цене, очистка строки поиска, переключение языка интерфейса, переход на страницу продавца и изменение количества товара в карточке. Проект выполнен на Java 17 с использованием Maven, JUnit 5, Selenide и AssertJ.

SUMMARY

This report presents the results of an educational practice project focused on UI test automation for the AliExpress web application. A checklist of 15 user scenarios was prepared, a Page Object Model architecture was designed, and five automated checks were implemented: sorting search results by price, clearing the search field, switching the interface language, opening a seller's shop page, and changing the product quantity on the product page. The project uses Java 17, Maven, JUnit 5, Selenide, and AssertJ.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	7
1.1 Формирование набора проверок	7
1.2 Автоматизированные сценарии	7
1.3 Результаты контрольного запуска	8
2 АРХИТЕКТУРА ТЕСТОВОГО ПРОЕКТА	11
2.1 Структура проекта и Page Object Model	11
2.2 Описание классов и методов	12
2.3 UML-диаграмма	13
3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ	15
3.1 Разработка вспомогательных классов элементов	15
3.2 Реализация теста сортировки результатов поиска	15
3.3 Реализация теста очистки строки поиска	16
3.4 Реализация теста переключения языка интерфейса	16
3.5 Реализация теста перехода на страницу продавца	17
3.6 Реализация теста выбора количества товара	17
3.7 Анализ выполненной части	18
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	19
5 ОТЗЫВ РУКОВОДИТЕЛЯ	21
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24

ВВЕДЕНИЕ

Интерфейсы современных веб-приложений часто обновляются: меняется разметка страниц, способ подгрузки данных и логика реакции на действия пользователя. Повторная ручная проверка одних и тех же сценариев отнимает много времени и повышает риск не заметить регрессионную ошибку. UI-автотестирование даёт возможность воспроизводимо выполнять пользовательские действия в браузере и автоматически сравнивать текущее состояние интерфейса с ожидаемым.

В качестве объекта проверки в рамках практики выбрано веб-приложение AliExpress. Для него характерны динамическая поисковая выдача, большое количество карточек товаров, кастомные элементы управления (не стандартные HTML-виджеты), всплывающие попапы и переходы между вкладками. Эти особенности делают систему подходящей для изучения методов стабильной автоматизации пользовательского интерфейса.

Цель практики состоит в разработке поддерживаемого набора UI-автотестов для проверки ключевых пользовательских сценариев AliExpress. Для достижения цели требовалось изучить возможности Selenide и JUnit 5, составить чек-лист, спроектировать Page Object Model, реализовать классы элементов и страниц, автоматизировать выбранные сценарии, а затем оценить результат контрольного запуска.

Работа выполнялась командой из четырёх студентов. Вклад участников распределялся следующим образом:

Степанов Павел, группа 4383, координировал работу команды, согласовывал структуру чек-листа и единые правила оформления результатов.

Шакуров Альберт, группа 4382, разработал основу автоматизированного проекта: настройку Maven, базовые классы BaseElement, BasePage, BaseTest, объекты четырёх основных страниц и пять первых UI-автотестов.

Кислица Сергей, группа 4388, прорабатывал сценарии работы с категориями, отзывами, окном «Поделиться», городом доставки.

Миридаштаки Фарид, группа 4343, реализовал сортировку результатов поиска по цене, очистку строки поиска, переключение языка интерфейса, переход на страницу продавца и изменение количества товара в карточке, а также добавил в общий проект вспомогательные классы элементов Item и Link.

Настоящий отчёт является индивидуальным и подробно раскрывает часть работы Миридаштаки Фарид: реализацию пяти автотестов (сортировка, очистка поиска, смена языка, страница продавца, счётчик количества), разработку новых классов элементов и страницы продавца, а также стабилизацию взаимодействия с динамическими элементами интерфейса.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Формирование набора проверок

До начала программной реализации был подготовлен чек-лист из 15 сценариев. Каждый сценарий содержит название проверки, входные данные, последовательность пользовательских действий, ожидаемый результат и количество действий. Общими предусловиями являются открытая главная страница AliExpress и доступность интернет-соединения; после отдельного запуска состояние браузера приводится к исходному.

Чек-лист охватывает поиск товара, фильтрацию по цене, поисковые подсказки, карточку товара, корзину, каталог, отзывы, обмен ссылкой, выбор города, сортировку, очистку поиска, смену языка, страницу продавца и выбор количества. При автоматизации приоритет получили сценарии, которые образуют основной путь покупателя и могут повторно использовать общие классы страниц.

1.2 Автоматизированные сценарии

В текущей версии проекта автоматизированы пять сценариев. Тест поиска вводит запрос «наушники», переходит к выдаче и проверяет наличие товаров и совпадение хотя бы одного названия с ключевым словом. Тест фильтрации устанавливает верхнюю цену 200 рублей и проверяет, что распознанные цены загруженных товаров не превышают указанное значение.

Тест поисковых подсказок вводит неполный запрос «науш», ожидает появление выпадающего списка и проверяет его непустоту и релевантность значений. Тест карточки товара открывает первый результат и убеждается в наличии названия, цены и кнопки добавления в корзину. Комплексный тест корзины ищет кабель USB, добавляет товар, изменяет количество до трёх и проверяет пересчёт общей стоимости.

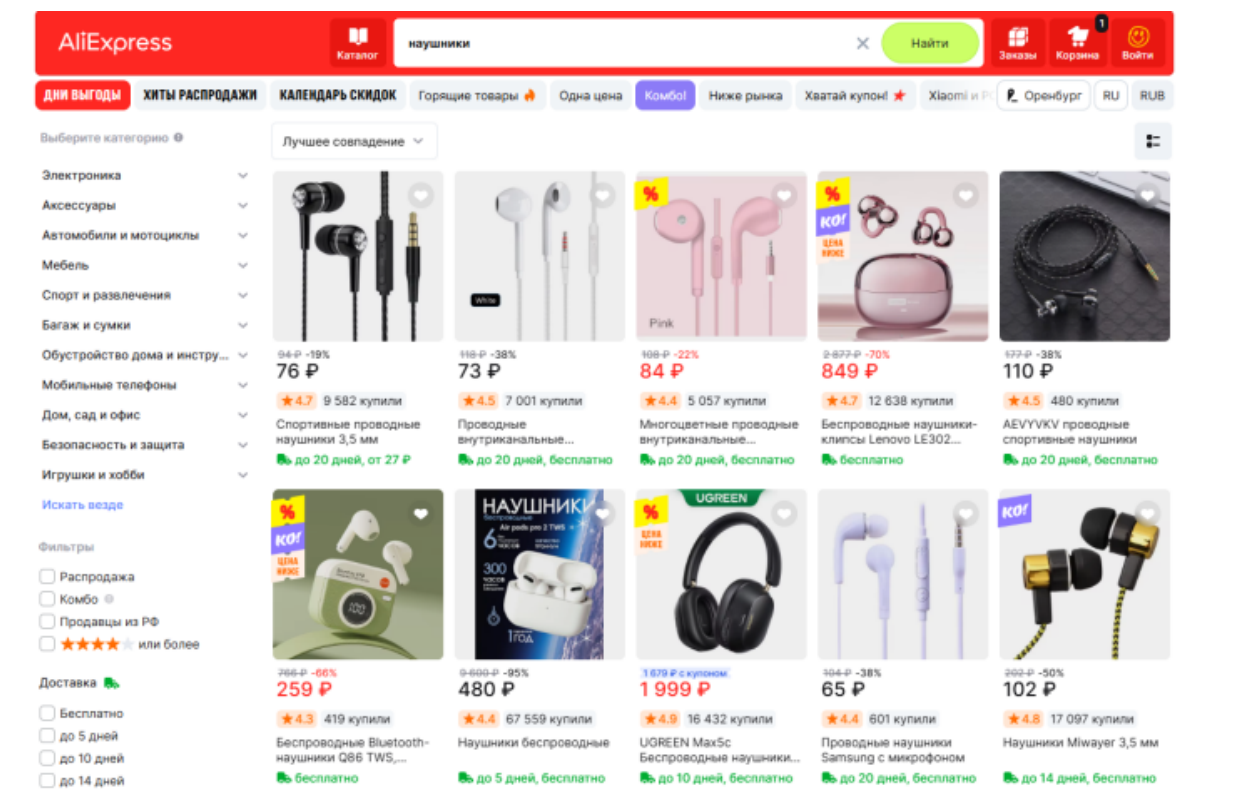


Рисунок 1 – Результат проверки поиска товара

На рис. 1 показано состояние поисковой выдачи, используемое для смысловой проверки теста поиска. Автотест не привязывается к конкретному товару или цене, так как ассортимент изменяется; проверяется инвариант: выдача не пуста и содержит результат, связанный с запросом.

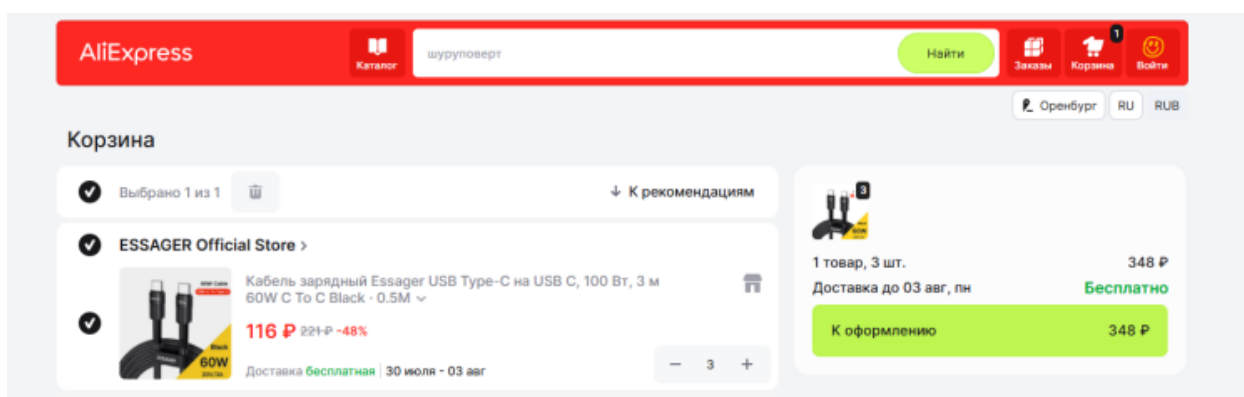


Рисунок 2 – Результат изменения количества товара в магазине

На рис. 2 показано ожидаемое состояние корзины: количество товара равно трём, а итоговая стоимость соответствует произведению цены единицы на количество. Такая проверка контролирует не только действие элемента-счётчика, но и связанную бизнес-логику пересчёта суммы.

1.3 Результаты контрольного запуска

Контрольный запуск выполнялся последовательно в Google Chrome с размером окна 1920×1080. Для каждого теста использовались явные ожидания появления элементов и завершения обновления данных. На рис. 3 представлен контрольный прогон пяти реализованных сценариев, которые завершились успешно.

```

[main] INFO com.codeborne.selenide.drivercommands.DisposablesRegistry - Add shutdown hook in 1 [size=1]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 37.87 s -- in tests.AddProductToCartAndChangeQuantityTest
[INFO] Running tests.FilterProductsByMaxPriceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.378 s -- in tests.FilterProductsByMaxPriceTest
[INFO] Running tests.OpenProductCardTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.446 s -- in tests.OpenProductCardTest
[INFO] Running tests.SearchSuggestionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.218 s -- in tests.SearchSuggestionsTest
[INFO] Running tests.SearchTest
[INFO] Найдено товаров: 20
[INFO] Первый товар: Спортивные проводные наушники 3,5 мм | Цена: 75 ?
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.981 s -- in tests.SearchTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO]
[INFO] Total time: 56.312 s
[INFO] Finished at: 2026-07-14T15:34:47+03:00
[INFO]

```

Рисунок 3 – Результаты контрольного запуска автотестов

Сводные результаты приведены в таблице 1.

Таблица 1 – Результаты выполнения автоматизированных тестов

№	Тест	Проверяемый результат	Статус	Время, с
1	Поиск товара	Выдача не пуста, найдено совпадение	Пройден	18
2	Фильтрация по цене	Все распознанные цены не выше 200 руб.	Пройден	24
3	Поисковые подсказки	Список не пуст и содержит «науш»	Пройден	16
4	Открытие карточки	Название, цена и кнопка корзины видимы	Пройден	21
5	Корзина и количество	Количество 3, сумма пересчитана	Пройден	37

Результат зависит от текущей версии внешнего сайта: AliExpress может изменять локаторы, показывать всплывающие попапы или защитную проверку (капчу). Поэтому тесты построены вокруг наблюдаемого поведения,

содержат динамические ожидания и учитывают несколько вариантов разметки для нестабильных элементов.

2 АРХИТЕКТУРА ТЕСТОВОГО ПРОЕКТА

2.1 Структура проекта и Page Object Model

Проект построен на паттерне Page Object Model, который отделяет логику взаимодействия с интерфейсом от логики самих тестов. Код разделён на три пакета:

`elements` — базовые элементы страницы (кнопки, поля ввода, ссылки). Каждый элемент инкапсулирует свой способ поиска и взаимодействия, поэтому тесты не зависят от структуры HTML напрямую.

`pages` — объекты, представляющие конкретные страницы или крупные части приложения (например, `MainPage` отвечает за поиск и навигацию, `ProductPage` — за карточку товара, `CartPage` — за корзину). Каждая страница содержит элементы и методы для доступных на ней действий.

`tests` — тестовые классы, написанные на уровне пользовательских действий. Тесты не содержат селекторов и не знают, как реализовано взаимодействие с элементами; они лишь вызывают методы страниц и проверяют результат через `AssertJ`.

Переходы между страницами реализованы через возврат соответствующих объектов: после поиска возвращается `SearchResultPage`, после открытия товара — `ProductPage`, после перехода в корзину — `CartPage`, после клика по продавцу — `SellerPage`. Это делает тесты читаемыми и приближенными к естественному описанию пользовательского сценария.

С недавнего изменения архитектуры браузер больше не открывается и не закрывается перед каждым отдельным тестом. Вместо этого `BaseTest` подключает `BrowserSessionExtension` — расширение JUnit 5, которое создаёт один браузер на весь тестовый запуск и закрывает его один раз в самом конце. Такое решение снижает частоту появления защитной проверки (капчи) AliExpress при массовом прогоне тестов и ускоряет общее время выполнения.

Тестовые классы также были перегруппированы по смысловому признаку вместо одного файла на тест: `SearchTests` (поиск, подсказки, сортировка, очистка поиска), `FilterTests` (фильтрация по цене), `ProductTests`

(карточка товара, отзывы, «Поделиться», продавец, количество), CartTests (добавление, изменение количества, удаление) и NavigationTests (категории, город доставки, язык интерфейса).

2.2 Описание классов и методов

Пакет `elements`. `BaseElement` — абстрактный базовый класс, содержащий общие методы `isDisplayed()` и `getText()`, а также защищённые методы `clear()` и `sendKeys()` для работы с полями ввода. От него наследуются `Button` (фабричные методы `byText()`, `byClass()`, метод `click()`), `Input` (методы `byName()`, `byId()`, `fill()`, `getValue()`), `Link` (метод `byClass()` и `clickViaJs()` для клика через JavaScript в случаях, когда обычный клик не срабатывает) и `Item` — универсальный класс для элементов без отдельной реализации (`div`, `span`, `li`), с фабричными методами поиска по классу, тексту и `data-testid`, а также методами `waitUntilHidden()` и `waitUntilTextChanges()` для динамического ожидания изменений.

Пакет `pages`. `BasePage` содержит методы `open()`, `getTitle()`, `refresh()` и `activatePage()` для восстановления фокуса окна после переходов между вкладками.

`MainPage` — главная страница: `search()`, `fillSearch()`, `clickClearSearch()`, `fillSearchAndWaitSuggestions()`, `switchLanguageToEnglish()` и `switchLanguageToRussian()`, `closeLocationPopupIfPresent()`, а также методы работы с каталогом (`openCatalog()`, `selectElectronicsCategory()`, `openAudioVideoCategory()`) и городом доставки (`openDeliveryAddress()`, `getDeliveryCity()`, `waitUntilDeliveryCityDisplayed()`).

`SearchResultPage` — поисковая выдача: `getProductPrices()`, `getProductTitles()`, `filterByMaxPrice()`, `sortByPriceAscending()`, `getSelectedSortType()`, `getSelectedSortText()`, `openFirstProduct()`.

`ProductPage` — карточка товара: `addToCart()`, `getCurrentQuantity()`, `incrementQuantity()`, `openCart()`, `openSellerPage()`, `openAllReviews()`, `openSharePopup()`.

CartPage — корзина: getProductTitle(), getProductQuantity(), getProductUnitPrice(), increaseProductQuantityTo(), waitTotalRecalculatedForQuantity(), getTotalPrice(), removeAllProducts(), isCartEmpty().

CategoryPage, DeliveryAddressModal, ReviewsPage, SharePopup — страницы каталога, смены города, отзывов и диалога «Поделиться», реализованные другими участниками команды.

SellerPage — страница магазина продавца: getShopTitle(), isShopTitleDisplayed().

Пакет tests. BaseTest хранит общие константы (поисковые запросы, размеры окна, таймауты) и настройку Chrome через метод configureSelenide(), вызываемую один раз при старте сессии через BrowserSessionExtension. Пять тестовых классов группируют 15 сценариев по смыслу, как описано в 2.1.

2.3 UML-диаграмма

Диаграмма классов проекта представлена на рисунке 4.

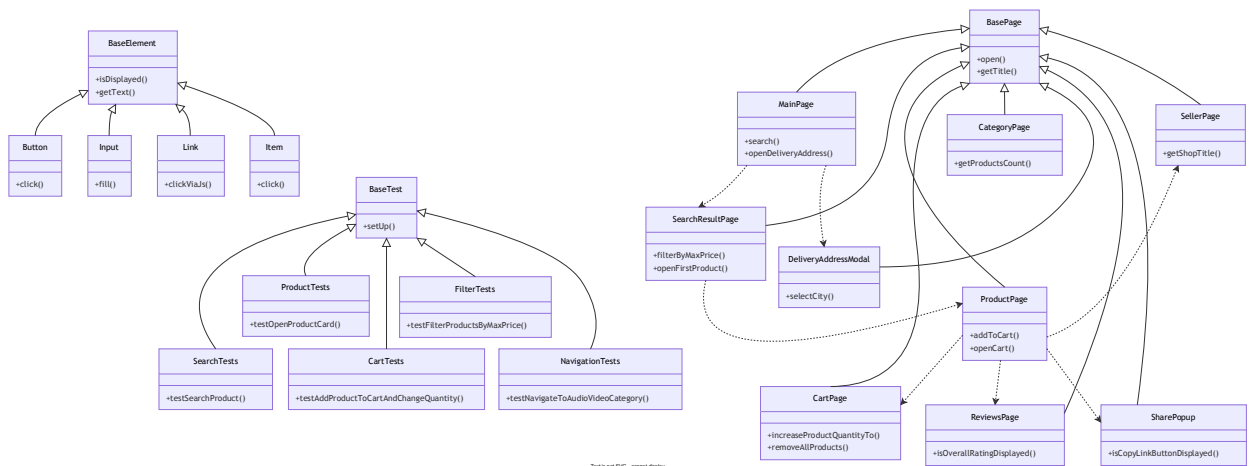


Рисунок 4 – UML-диаграмма классов тестового проекта

На диаграмме видно, что все элементы наследуются от абстрактного класса BaseElement, а все страницы — от BasePage. Тесты наследуются от BaseTest, который через BrowserSessionExtension управляет единой сессией браузера на весь запуск. Переходы между страницами показаны стрелками, отражающими возврат соответствующих объектов при выполнении действий.

Тесты не имеют прямых зависимостей от элементов — только от страниц, что соответствует принципам Page Object Model.

3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ

В рамках учебной практики мной были выполнены следующие задачи: реализация пяти автоматизированных тестов (№11–15), разработка вспомогательных классов элементов Item и Link, создание класса страницы SellerPage, а также участие в переносе тестов в сгруппированные по смыслу классы SearchTests, NavigationTests и ProductTests в рамках общей доработки архитектуры по замечаниям руководителя практики.

3.1 Разработка вспомогательных классов элементов

Для реализации тестов №14 и №13 потребовались элементы, которых не было в проекте на момент начала работы: универсальный элемент для div, span, li без отдельной специализации и ссылка с кликом через JavaScript. Класс Item получил фабричные методы поиска по CSS-классу, по классу и тексту одновременно, а также по data-testid внутри конкретного контейнера с исключением совпадающих скрытых блоков разметки. Класс Link получил метод clickViaJs() для случаев, когда стандартный клик Selenide не срабатывает из-за особенностей обработки события на сайте.

Для подтверждения перехода на страницу продавца создан новый класс страницы SellerPage с методами getShopTitle() и isShopTitleDisplayed().

3.2 Реализация теста сортировки результатов поиска

На основе чек-листа мной был реализован тест №11 — «Сортировка результатов поиска по цене».

Данный тест проверяет, что после выбора сортировки «Сначала дешёвые» товары в выдаче действительно переупорядочиваются по возрастанию цены.

Основные шаги теста:

1. Открыть главную страницу AliExpress.
2. Выполнить поиск по запросу «рюкзак».
3. Открыть выпадающий список сортировки и выбрать «Сначала дешёвые».
4. Проверить, что на странице отображается не менее двух товаров с ценой.

5. Проверить, что в URL применён параметр `price_asc`.
6. Проверить, что в списке сортировки отображается выбранный пункт «Сначала дешёвые».

Тест был реализован с использованием следующих классов: `MainPage` — открытие главной страницы; `SearchResultPage` — применение сортировки и чтение цен.

Тест не привязывается к конкретному товару, так как ассортимент сайта изменяется — проверяется, что сортировка действительно применена, а не конкретные значения цен.

3.3 Реализация теста очистки строки поиска

На основе чек-листа мной был реализован тест №12 — «Очистка строки поиска».

Данный тест проверяет, что кнопка очистки полностью удаляет предыдущий запрос из поля, а не только скрывает его визуально.

Основные шаги теста:

1. Открыть главную страницу AliExpress.
2. Ввести в поле поиска запрос «наушники».
3. Нажать кнопку очистки поля (крестик).
4. Ввести новый запрос «смартфон».
5. Проверить, что в поле отображается только новый запрос.
6. Проверить, что старый запрос отсутствует.

Тест был реализован с использованием класса `MainPage` — заполнение поля, очистка и чтение текущего значения.

3.4 Реализация теста переключения языка интерфейса

На основе чек-листа мной был реализован тест №13 — «Переключение языка интерфейса».

Данный тест проверяет, что переключение языка в шапке сайта действительно меняет текст интерфейса на английский.

Основные шаги теста:

1. Открыть главную страницу AliExpress.

2. Переключить язык интерфейса на английский через переключатель в шапке сайта.
3. Обновить страницу для применения изменений.
4. Проверить, что текст кнопки поиска изменился с «Найти» на «Find».

Тест был реализован с использованием класса MainPage. После завершения теста язык автоматически возвращается на русский методом `resetLanguage()`, выполняемым после каждого теста в `NavigationTests`, чтобы не влиять на тесты, ищущие элементы по русскому тексту.

3.5 Реализация теста перехода на страницу продавца

На основе чек-листа мной был реализован тест №14 — «Проверка ссылки на продавца».

Данный тест проверяет корректность перехода со страницы товара на страницу магазина продавца.

Основные шаги теста:

1. Открыть главную страницу AliExpress.
2. Выполнить поиск по запросу «наушники».
3. Открыть карточку первого товара.
4. Нажать на название магазина.
5. Проверить, что название магазина отображается на странице продавца.
6. Проверить, что название магазина не пустое.

Тест был реализован с использованием классов: `ProductPage` — переход к продавцу; новый класс `SellerPage` — проверка заголовка магазина. Изначальный пункт чек-листа предполагал два клика, однако проверка на реальном сайте показала, что в десктопной версии достаточно одного клика по ссылке с названием магазина.

3.6 Реализация теста выбора количества товара

На основе чек-листа мной был реализован тест №15 — «Проверка выбора количества товара».

Данный тест проверяет, что счётчик количества товара корректно обновляется после нажатия кнопки «+».

Основные шаги теста:

1. Открыть главную страницу AliExpress.
2. Выполнить поиск по запросу «наушники».
3. Открыть карточку первого товара.
4. Добавить товар в корзину.
5. Проверить, что счётчик показывает «1».
6. Нажать «+» для увеличения количества.
7. Проверить, что счётчик показывает «2».

Тест был реализован с использованием класса `ProductPage` — добавление в корзину, чтение и изменение значения счётчика. Отдельного поля количества на странице не существует до добавления товара в корзину — счётчик появляется на месте кнопки только после первого клика.

3.7 Анализ выполненной части

Пять реализованных тестов расширили общую архитектуру проекта, не изменив уже написанный код остальных участников команды: тесты были добавлены как новые методы в сгруппированные классы `SearchTests`, `NavigationTests` и `ProductTests`, а два новых класса элементов (`Item`, `Link`) и класс страницы `SellerPage` — как отдельные файлы. Основные технические сложности были связаны с нестандартными элементами управления AliExpress: кастомным дропдауном сортировки вместо обычного `select`, визуально скрытыми радиокнопками внутри опций сортировки, скрытой sticky-копией счётчика количества и `cookie`, сохраняющей выбранный язык между тестами. Во всех случаях вместо статических пауз применялись целевые динамические ожидания, что соответствует требованиям руководителя практики по построению устойчивых проверок.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения практической работы использовался следующий стек технологий и инструментов; объектом автоматизации являлся сайт AliExpress [1]:

Java 17 [2] — язык программирования, на котором написаны все классы элементов, страниц и тестов. Выбор версии обусловлен статусом LTS (Long-Term Support), обеспечивающим стабильность и долгосрочную поддержку.

Maven [3] — система управления зависимостями и сборки проекта. Позволяет централизованно описывать все необходимые библиотеки в файле `pom.xml` и обеспечивать единообразную сборку на всех компьютерах команды.

JUnit 5 [4] — фреймворк для написания и запуска тестов. Использовались аннотации `@Test` для тестовых методов и `@AfterEach` для действий после выполнения теста (например, сброса языка интерфейса в `NavigationTests`). Настройка браузера вынесена из `@BeforeEach` в отдельное расширение `BrowserSessionExtension`, которое создаёт один браузер на весь тестовый запуск вместо нового браузера перед каждым тестом.

Selenide [5] — фреймворк для UI-автоматизации, построенный на основе Selenium WebDriver. Обеспечивает управление браузером, поиск элементов и встроенные динамические ожидания. Использовался для взаимодействия со всеми элементами страниц.

Selenium WebDriver [6] — низкоуровневая работа с окнами браузера и явными ожиданиями (`WebDriverWait`) в случаях, где встроенных ожиданий Selenide было недостаточно, например при ожидании применения фильтра по цене.

AssertJ [7] — библиотека для читаемых проверок. Позволяет писать проверки в формате `assertThat(actual).as("описание").isEqualTo(expected)`, что делает код тестов понятным и информативным при падении.

Git и GitHub — система контроля версий и платформа для хостинга репозитория. Каждый участник команды работал в отдельной ветке, делал коммиты и отправлял изменения в общий репозиторий.

IntelliJ IDEA — среда разработки, используемая всеми членами команды для написания кода, рефакторинга и запуска тестов.

Выбранный набор технологий является стандартным для UI-автотестирования на Java. Java и Maven обеспечивают воспроизводимую структуру проекта, JUnit организует тестовый жизненный цикл, Selenide сокращает объём технического кода благодаря встроенным ожиданиям, а AssertJ делает проверки понятными при чтении отчёта о падении. Git и GitHub обеспечивают удобную командную работу над общим репозиторием.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики была достигнута поставленная цель — разработан набор UI-автотестов для проверки ключевых пользовательских сценариев веб-приложения AliExpress. Все задачи, определённые во введении, были решены.

Разработан и согласован с руководителем чек-лист, включающий 15 пользовательских сценариев, охватывающих поиск, фильтрацию, работу с карточкой товара, корзиной, категориями, отзывами, городом доставки и другими элементами интерфейса. Чек-лист оформлен в виде таблицы и загружен в репозиторий проекта.

Спроектирована архитектура тестового проекта на основе паттерна Page Object Model. Код разделён на три слоя: элементы (elements), страницы (pages) и тесты (tests). По замечаниям руководителя практики тестовые классы были дополнительно сгруппированы по смысловым блокам (SearchTests, FilterTests, ProductTests, CartTests, NavigationTests), а настройка браузера вынесена в отдельное расширение BrowserSessionExtension, создающее одну сессию браузера на весь тестовый запуск. Такая структура снижает частоту появления защитной проверки сайта и упрощает поддержку проекта при изменении вёрстки.

На языке Java с использованием фреймворков Selenide, JUnit 5 и AssertJ реализованы классы базовых элементов (Button, Input, Link, Item), страниц (MainPage, SearchResultPage, ProductPage, CartPage, CategoryPage, DeliveryAddressModal, ReviewsPage, SharePopup, SellerPage) и пяти сгруппированных тестовых классов. Всего реализовано 15 автоматизированных тестов, покрывающих основные сценарии работы пользователя, с общим количеством действий не менее 50.

Налажена командная работа с распределением задач между четырьмя участниками и контролем качества кода. Использование Git и GitHub позволило эффективно управлять изменениями и интегрировать результаты работы всех членов команды.

В индивидуальной части реализованы пять тестов (№11–15): сортировка результатов поиска по цене, очистка строки поиска, переключение языка интерфейса, переход на страницу продавца и изменение количества товара в карточке. Дополнительно разработаны вспомогательные классы элементов Item и Link, а также новый класс страницы SellerPage.

Выполнен контрольный запуск всех тестов. Тесты успешно проходят, подтверждая корректность работы автоматизированных проверок. Результаты запуска зафиксированы и представлены в отчёте.

Таким образом, задачи практики выполнены в полном объёме. Разработанный проект может служить основой для дальнейшего расширения набора тестов и интеграции в процессы непрерывной интеграции. В ходе работы получены практические навыки автоматизации UI-тестирования, командной разработки на общем репозитории и построения устойчивых проверок для сайтов с динамическим интерфейсом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. AliExpress [Электронный ресурс]. URL: <https://aliexpress.ru/> (дата обращения: 06.07.2026).
2. Java Platform, Standard Edition 17 API Specification [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/> (дата обращения: 07.07.2026).
3. Apache Maven Guides [Электронный ресурс]. URL: <https://maven.apache.org/guides/> (дата обращения: 09.07.2026).
4. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 09.07.2026).
5. Selenide Documentation [Электронный ресурс]. URL: <https://selenide.org/documentation.html> (дата обращения: 09.07.2026).
6. Selenium WebDriver Documentation [Электронный ресурс]. URL: <https://www.selenium.dev/documentation/webdriver/> (дата обращения: 11.07.2026).
7. AssertJ Core Documentation [Электронный ресурс]. URL: <https://assertj.github.io/doc/> (дата обращения: 11.07.2026).